

OILTRACE

Finding marine oil spills from satellites — and working out which ship is responsible.
A plain-English guide to what the system predicts, who uses it, and why it matters.

In one paragraph

A satellite radar image of the sea arrives. Somewhere in it is a dark patch. OILTRACE decides whether that patch is **mineral oil** or a harmless look-alike; estimates **how much oil and how old** it is; runs the ocean backwards to work out **where and when it was released**; runs it forwards to say **which coastline it will hit and when**; and then matches the reconstructed release against ship movement records to produce a **ranked list of suspect vessels, each with its reasons written out**. It finishes in about 49 seconds on an ordinary laptop, with no GPU.

Measured performance

What	Measured result	How it was measured
Attribution accuracy	100% top-1 (10/10 scenes)	Ground truth known; culprit is the top-ranked ship
Nearest-ship baseline	0% top-1, 30% top-3	Same scenes, ranked by distance alone
Slick outline quality	F1 0.813 · IoU 0.699	Against the known truth mask
Release-point error	9.4 km (vs 13.9 km)	Source inversion vs backward-cloud peak
Release-time error	3.3 h	Estimated vs true discharge start
End-to-end runtime	49 s	Laptop CPU, no GPU

Source: data/validation.json — 10 independent scenes with known ground truth. Every figure in this document is regenerated from the repository; none is hand-written.

1 · The problem, in plain terms

Most oil in the sea does not come from dramatic tanker accidents. It comes from routine, deliberate discharges: a ship cleaning its tanks or pumping out oily bilge water while under way, usually at night, usually far from shore. Each release is small compared with a shipwreck. There are a great many of them.

Satellites can already see these slicks. Oil flattens the tiny ripples that radar bounces off, so a slick shows up as a **dark patch** on a radar image, day or night, through cloud. That part is largely solved.

The gap this project addresses

Seeing the slick is not the same as knowing **who did it**. By the time an image is analysed, the ship has sailed on and the slick has drifted kilometres from where it was released. Existing operational services are very good at raising the alert and then hand the question of responsibility to a human analyst. Without a named vessel there is usually no case, no penalty, and no deterrent — so the discharges continue.

OILTRACE is built for that second question. It treats **attribution** — not detection — as the hard part, and is designed so that its answer can be checked, argued with, and rejected.

2 · What the model actually predicts

The system is not one model. It is a chain of five, each answering a question a human investigator would ask next.

#	Question	What is predicted	Form of the answer
1	Is that dark patch oil?	Oil vs look-alike	P(oil), 0 to 1, per region
2	How bad is it?	Thickness, volume, tonnes, age	Bonn class + ranges
3	Where did it come from?	Release point, time, course, speed	A source track, not a dot
4	Where is it going?	Drift for the next 18 hours	Landfall time + location
5	Who did it?	Ranked suspect vessels	Score, verdict, written reasons

The prediction that matters most

Stages 1 and 2 describe the slick. Stage 5 is the product. The output for each candidate ship is a score built from **six independent evidence axes**, so that no single coincidence can convict a vessel:

Evidence axis	Weight	In plain English
Source-track match	3.2	Was the ship on the reconstructed release line, at the release time?
Origin envelope	2.4	Did its track pass through the likely origin area at all?
Behaviour	1.6	Did it slow down, alter course, or loiter during the window?
Dark period	1.2	Did its transponder go silent during the window?
Slick alignment	1.0	Does the slick lie along the ship's heading?

Evidence axis	Weight	In plain English
Vessel prior	0.7	Is it a type that carries oil (tanker, bulk)?

The weights are in `sagar/core/attribute.py`. Time is weighted highest deliberately: being in the right place is weak evidence — shipping lanes are busy — but being in the right place *at the right time, on the right heading* is not.

3 · Who uses it, and how

The intended users are the agencies that already hold this responsibility. The system is designed to slot into their existing workflow rather than replace it.

User	What they get	What they do with it
Coast Guard operations room	Alert with severity, position, area, jurisdiction	Decide whether to launch a Dornier or a ship, and where to send it
Pollution response officer	18-hour drift forecast, landfall time, nearest coast	Pre-position booms and shoreline teams before the oil arrives
Enforcement / investigation cell	Ranked suspects with per-axis reasons and an evidence pack	Request port-state inspection; take an oil sample for fingerprinting
Legal / prosecution	Evidence pack: JSON, GeoJSON, CSV, PDF, provenance chain	Build the case file; show the method was auditable
Maritime board / state authority	Jurisdiction and MARPOL Special Area classification	Establish which legal regime and penalty applies

A worked example — the flow in practice

Taken from incident OIL - 2026 - 1359, one of the 7 incidents currently in the repository:

- **Detected.** A 120 km² oil-like feature in the Gulf of Kutch, 4 km from the Kutch coastline. Severity **CRITICAL**.
- **Characterised.** Bonn class 'metallic'; roughly 1,600 tonnes; about 8 hours old, with the uncertainty stated rather than hidden.
- **Traced back.** Release corridor reconstructed to a window 7.4 to 6.6 hours before the image.
- **Projected forward.** Drift run 18 hours ahead against the coastline to give a landfall estimate.
- **Attributed.** MT KAVERI STAR, a 244 m tanker, ranked top: it passed through the origin envelope, dropped 57% below its own median speed, altered course 17°, and went AIS-silent for 44 minutes inside the release window.
- **Tasked.** A P1 patrol action is generated — observe the source corridor on the next satellite pass.

Note the shape of that output. It is not "this ship is guilty". It is **four independent reasons**, each traceable to a data source, that an officer can read in fifteen seconds and challenge individually.

4 · Why this matters in real life

For the coastline and the people on it

An oil slick that reaches shore stops being a remote-sensing problem and becomes an economic one. Mangroves, fishing grounds, salt pans, desalination intakes and tourist beaches all take damage that is expensive and slow to reverse. Cleanup at the shoreline costs far more than containment at sea, so **hours of warning are worth a great deal**. The 18-hour drift forecast exists for exactly this: it tells a responder which stretch of coast to defend, before the oil is there.

For enforcement

A polluter who is never identified pays nothing, and the discharge is repeated. The chain that has to hold is: **detect** → **attribute** → **inspect** → **sample** → **prosecute**. Detection alone breaks that chain at step two. By producing a named, reasoned shortlist while the ship is still at sea, the system makes the next physical step — a port-state inspection and an oil sample for fingerprinting — possible at all.

For deterrence

This is the real prize. Deliberate discharge is a calculated decision: cheaper than paying for port reception facilities, and historically very unlikely to be punished. Changing the perceived probability of being caught changes that calculation for every ship on the route, including the ones never actually inspected.

What the system is, and is not

Every output is an **investigative lead**, not proof. A conviction for illegal discharge normally requires chemical fingerprinting of an oil sample taken from the suspect vessel and matched to a sample from the slick. OILTRACE tells an officer *which ship to board and which 40 minutes to ask about*. It does not, and should not, replace that step. The PDF this system generates says so on its cover page.

5 · Four design decisions worth defending

1. Inverting the source, not the cloud

The obvious way to find where a slick came from is to run the drift backwards and see where the particles converge. That works for a single instantaneous release. It fails for the case that actually matters: a ship discharging while under way lays down a **line** of oil over tens of minutes. That line was already long at the moment of release, so running it backwards never collapses it to a point.

So the system inverts the **source** instead: it guesses a moving line release — start time, duration, course, speed, position — pushes that guess forward through the same ocean, and scores the resulting footprint against the observed slick. The measured payoff is direct:

Method	Mean error against true release point
Backward-cloud peak (the obvious approach)	13.9 km
Source-term inversion (this system)	9.4 km

Beyond the smaller error, inversion returns something a probability blob cannot: a **candidate track with timing**, which is what makes matching against ship movements in time — not just space — possible.

2. Refusing to convict the nearest ship

The lazy baseline is to blame whichever vessel was closest. The test set is built specifically to punish that: it includes decoys that are in the right place at the wrong time, the right time in the wrong place, and innocent ships transiting straight through the origin.

Method	Top-1 correct	Top-3 correct
Nearest-vessel baseline	0%	30%
Six-axis attribution (this system)	100%	100%

The baseline is not a straw man — proximity is what a human analyst reaches for first. It scores 0% here because the decoys are designed to defeat it.

3. Abstaining instead of guessing

A system that always names someone is useless, because it names someone even when it has nothing. Each candidate must clear three separate bars — enough evidence, good enough AIS to mean anything, and support from more than one axis — or it is returned as **INSUFFICIENT_EVIDENCE** with the reason stated. It stays visible to the analyst; it is simply not an accusation.

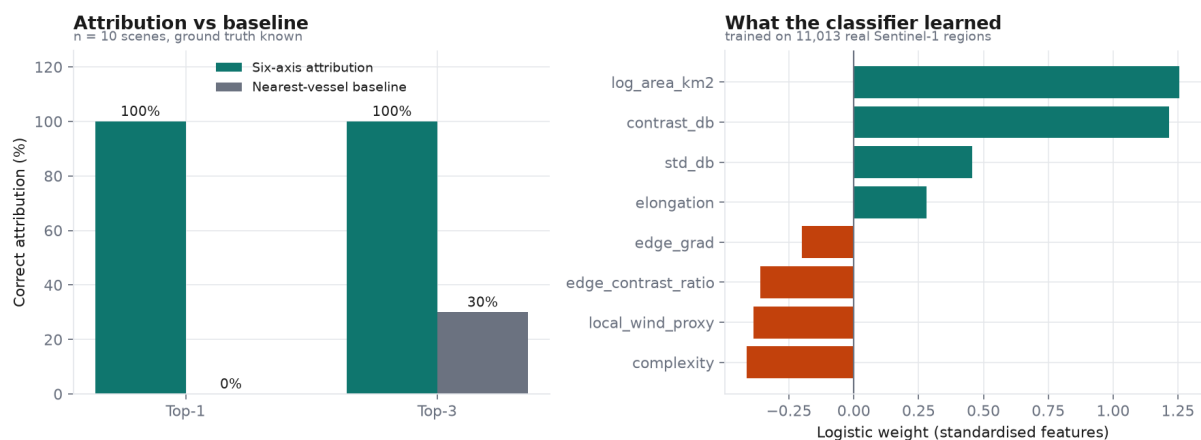
Across the 7 incidents currently in the repository, **12 of 37 candidates (32%) are withheld** this way rather than ranked.

4. Saying which ships it cannot see

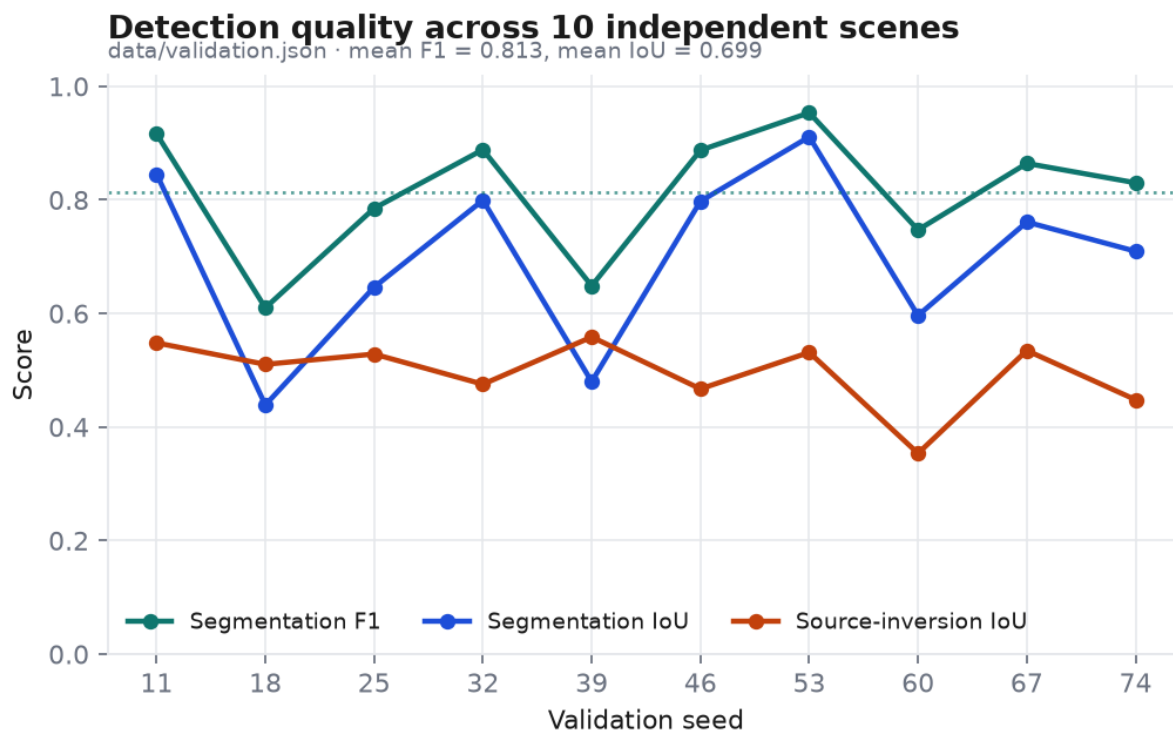
A vessel that switches its transponder off during a discharge is invisible to any AIS-only method — and switching it off is exactly what a deliberate polluter does. The system therefore looks for bright ship-like targets in the radar image itself, so a vessel with no AIS record still appears as a candidate. The transponder gap is scored as **evidence**, not treated as missing data.

6 · Results, including the ones that are not flattering

All five charts below are generated from this repository's own outputs by `scripts/make_figures.py`. Nothing is illustrative.



Left: attribution against the nearest-ship baseline on 10 scenes with known ground truth. Right: the eight features the slick classifier learned, from 11,013 real Sentinel-1 regions.



Slick-outline quality per scene. Mean F1 0.813, mean IoU 0.699. The variation across scenes is real and is not smoothed away.

Oil vs look-alike confusion matrix

acc 0.52 · precision 0.51 · recall 1.00 · F1 0.67 | real-SAR model on synthetic scenes (domain gap)

Actual	Predicted	
	look-alike	oil
look-alike	2 TN	30 FP
oil	0 FN	31 TP

Oil vs look-alike on 63 held-out regions. Recall is perfect — it misses no real slick — but precision is poor.

An honest limitation, stated plainly

The slick classifier is trained on **real** Sentinel-1 imagery (11,013 regions, Zenodo 8346860) and on its own real-data test split it scores **AUC 0.959, accuracy 0.923**. The confusion matrix above scores that same model against **synthetic** scenes, where it reaches only 0.52 accuracy: it catches every true slick but flags most look-alikes as oil too.

This is a **domain gap** — synthetic look-alikes do not reproduce the radar statistics of real ones — and not a coding error; the scoring path was verified identical to the live pipeline to within 2.2e-16. It is reported as measured rather than replaced with a more flattering split, because a system whose stated purpose is to abstain honestly cannot hide its own weakest number.

The practical consequence is the correct one for this application: at this operating point the system **over-refers rather than misses**. For a screening tool that feeds human review, a false alarm costs an analyst a few minutes; a missed spill costs a coastline.

7 · What already exists, and where the gap is

This section is the honest competitive review. Every claim below is sourced; where a source could not be verified, that is said instead of guessed at.

India: INCOIS Online Oil Spill Advisory (OOSA)

A correction worth making first. It is sometimes said that India's oil spill system has not been updated since 2019. That is **not correct** and should not be claimed. OOSA was established in 2015, and the INCOIS Annual Report 2023-24 records that it was **upgraded to Version 5.0 with advanced GIS and predictive capabilities**. The service is live today. A project that argues against a straw man is easy to dismiss.

The real gap is not staleness — it is **scope**. OOSA is a **trajectory forecasting** tool: it wraps NOAA's GNOME model, forced by INCOIS ocean currents and ECMWF/NCMRWF winds, and predicts where oil will drift. The operator must **type in** the spill's location, time, pollutant type and quantity to get a forecast.

That means OOSA answers question 4 of the five in Section 2. It does not detect the slick from satellite imagery, and it does not attribute it to a vessel. A search of the INCOIS Annual Report 2023-24 for 'AIS', 'automatic identification', 'attribution', 'polluter' and 'synthetic aperture' returns no relevant results.

No **operational, continuously-running** national satellite oil spill detection service run by ISRO/NRSC could be verified. Oil spill mapping over Indian seas appears in the literature as research, and one-off detections are documented — EOS-04 detected a spill off the Kerala coast on 27 May 2025. This is stated as an **absence of evidence**, not proof of absence.

Europe: EMSA CleanSeaNet — the closest operational analogue

CleanSeaNet is the system this project should be measured against. Operational since April 2007, it processes over 3,000 satellite images a year and delivers **analysed imagery to national contact points in under 30 minutes** of acquisition. In 2016 it flagged 1,586 high-confidence and 1,582 lower-confidence spills. Detections per million km² roughly halved between 2007 and 2017 — evidence that this kind of surveillance does deter.

On attribution, EMSA is precise about what it does: analysis is **semi-automatic, with trained operators**, combining imagery with AIS and LRIT to offer *potential polluter identification*. Verification is explicitly left to Member States, because a detection may be sewage, algae or upwelling rather than mineral oil. **Attribution is not automated end to end and carries no legal finding.**

Response models: GNOME, ERMA, OpenDrift/OpenOil

NOAA's GNOME (the engine inside OOSA) and ERMA, and MET Norway's OpenDrift/OpenOil — in daily operational use and the reference implementation for oil drift — are all **drift and response** tools. None performs detection or attribution. This project implements the same transport physics directly so it installs from a handful of pure-Python wheels and runs offline.

The one system that does automate attribution: SkyTruth Cerulean

Cerulean, launched in 2023, is the honest closest prior art: a U-Net on Sentinel-1 imagery, with vessels ranked against each slick by track parallelism, proximity and timing over an AIS window of −8 h to +6 h. Its reported accuracy is **92% top-5 and 76% top-1** against known coincident vessels.

Two things follow. First, the approach is sound — an independent group reached a similar architecture. Second, Cerulean is an **NGO transparency tool, not an enforcement system**, it is not deployed for Indian waters, and SkyTruth themselves note that slicks cannot be definitively identified from SAR alone.

Comparing accuracy honestly

This project reports 100% top-1 on 10 scenes with known ground truth. Cerulean reports 76% top-1 on real satellite data. **These numbers are not comparable.** Ten controlled scenes with simulated AIS is a far easier test than the open ocean, and the honest reading is that this system has demonstrated its *method*, not yet its *field accuracy*. Cerulean's figure is the benchmark to be measured against once real imagery and real AIS are wired in.

The enforcement gap — the strongest argument for this work

The case for attribution does not rest on opinion:

- **About 80% of confirmed oil slicks never have a polluter identified.** Bonn Agreement aerial surveillance report, 2008 — the single clearest statement that detection alone does not produce accountability.
- **76% of successful US MARPOL prosecutions came from whistleblowers**, not from surveillance — National Whistleblower Center analysis of 100 prosecutions, 1993–2017. Cases are made by human informants, not remote sensing.
- **SkyTruth found no case where satellite imagery was decisive** in a bilge dumping prosecution; it served only to corroborate.
- **Oil disperses within roughly 12 hours of discharge**, which is the physical reason late attribution fails and why speed matters.
- The Bonn Agreement excludes follow-up outcomes from its reports because proceedings “may take a year or more”.

Where this project sits

System	Detects	Forecasts drift	Attributes to a vessel
INCOIS OOSA (India, v5.0)	No — manual input	Yes (GNOME)	No
EMSA CleanSeaNet (EU)	Yes	Partly	Semi-automatic; operator-led, no legal finding
NOAA GNOME / ERMA (US)	No	Yes	No
OpenDrift / OpenOil (NO)	No	Yes	No
SkyTruth Cerulean (NGO)	Yes (U-Net)	No	Yes — 92% top-5 / 76% top-1
OILTRACE (this project)	Yes	Yes	Yes — with reasons, and it abstains

The distinguishing claim is narrow and defensible: **no system in operational use for Indian waters combines detection, drift and reasoned vessel attribution in one pipeline**, and none of the attribution systems that do exist state, per candidate, why they concluded what they did or when they have too little evidence to conclude anything.

Sources: INCOIS OOSA portal and Annual Report 2023-24; EMSA CleanSeaNet service pages and the EMSA 10-year review (Interspill); Bonn Agreement aerial surveillance report 2008; National Whistleblower Center APPS analysis; SkyTruth Cerulean methods; Dagestad et al., OpenDrift v1.0, Geosci. Model Dev. 11, 2018. Figures for annual illegal discharge volumes in Indian waters and global MARPOL conviction rates were sought and could not be verified from a primary source; they are deliberately not quoted here.

8 · Limitations, stated up front

These are the things a reviewer would find, listed here so they are found here first.

Limitation	Why it matters	Route out
Metoccean fields are analytic in the demo	Real currents and wind have structure that makes inversion harder; the quoted errors would grow	CMEMS + ERA5 adapters already exist in loaders.py
Single polarisation (VV intensity only)	Polarimetric features are the standard next lever for separating oil from look-alikes	Add VH and entropy/alpha features
Classifier over-refers on look-alikes	0.52 accuracy on synthetic held-out data; over-refers rather than misses	Train and evaluate on the Zenodo split in its own domain
U-Net wired but not trained (logistic-8feature is what runs)	The deep segmenter is integrated behind a fallback, but no checkpoint ships, so the logistic detector is what produced every result here	gpu/ DGX kit trains it; pipeline picks it up automatically
No land or ice masking	Coastal land and ice can produce dark regions	Standard coastline mask at the preprocessing stage
AIS is simulated in the demo scenarios	Real AIS is sparser, noisier and has genuine coverage gaps	Live AIS ingest endpoints are implemented and wired

Two of these are deliberate scope choices rather than defects. The physics engine is implemented directly instead of depending on OpenDrift so the whole system installs from a handful of pure-Python wheels and runs offline on a laptop — which is what a demonstration in a hall without internet requires.

9 · What would make this operational

Ordered by how much each would add, relative to the effort:

- **Train the U-Net on the Zenodo set.** The integration is already written and falls back safely; only the checkpoint is missing. This directly attacks the weakest measured number in the document.
- **Swap in real CMEMS currents and ERA5 wind.** The adapters exist. This converts every drift and inversion figure from indicative to operational.
- **Ingest live AIS for the Indian EEZ.** The endpoints are built; what is needed is a feed agreement with the relevant authority.
- **Validate against a known real incident.** One documented spill with a confirmed responsible vessel, run end to end, is worth more than any amount of synthetic validation.
- **Add polarimetric features.** The standard next lever for oil versus look-alike separation.
- **Calibrate the scores.** Turn the attribution score into a stated probability an officer can reason about, with an accuracy-versus-referral curve to set the threshold deliberately.

The one-line case for this project

Satellite oil-spill **detection** is a solved and deployed problem. Automated, reasoned, auditable **attribution** — naming the ship while it is still at sea, and being explicit about when it cannot — is the step that turns a detection into an enforcement action, and that is the step this project builds.

Appendix · Where every number came from

The document is generated from the repository, so each figure can be traced to a file and regenerated.

Claim in this document	Source in the repository
100% attribution, baseline 0%	data/validation.json (n=10)
F1 0.813, IoU 0.699	data/validation.json
Inversion 9.4 km vs PDF peak 13.9 km	data/validation.json
Runtime 49 s	data/validation.json
26 detections across 7 incidents	data/out/*/report.json
12 of 37 candidates withheld	data/out/*/report.json (verdict field)
Classifier AUC 0.959, acc 0.923, N=11,013	sagar/data/oil_classifier.json
Confusion matrix (63 held-out regions)	scripts/eval_classifier.py → data/classifier_eval.npz
All five charts	scripts/make_figures.py → figures/
Six evidence axes and weights	sagar/core/attribute.py
Abstention rules	sagar/core/attribute.py :: verdict_for()

Rebuild this PDF: `.venv/bin/python scripts/make_report.py`